

Complete LED Sequence System - Final Summary

Architecture Overview

The system is now cleanly separated into:

1. **Auto-generated files** - Model data (tedious, generated from source)
2. **Hand-coded files** - Sequence choreography (creative, custom timing)
3. **Framework files** - Generic infrastructure (no model-specific references)

File Organization

Auto-Generated Files (Model Data)

hdlo_models.h

- Model data array declarations
- `nummodels` constant
- `modelNames[]` array declaration

hdlo_models.cpp

- Model data arrays (120x6 integers per model)
- `modelNames[]` definition
- `initializemodels()` function

Hand-Coded Files (Choreography)

sequences.h

- Simple header declaring `initializeSequences()`

sequences.cpp

- All sequence definitions
- Creative timing and transitions
- References models via `modelNames[]`

Framework Files (No Changes Needed)

modelsequence.h / modelsequence.cpp

- Generic sequence management
- Registry system
- Transition handling

hdlo_controller.ino

- Main program loop
- Calls initialization functions

models.h / models.cpp

- Base color model classes

colorfunctions.h / colorfunctions.cpp

- Color function definitions

Other files

- `ledconstants.h`, `edgesetup.h`, `teensy4controller.h` (unchanged)
-

Key Features

Variable-Length Color Arrays

- ✔ Support for 1-6 color functions per step

```
cpp

seq->addStepByName(modelNames[0], {"dark"}, 5000, FADE, 1000); // 1 color
seq->addStepByName(modelNames[1], {"dark", "rainbow", "bluetored"}, 5000, FADE, 1000); // 3 colors
seq->addStepByName(modelNames[2], {"dark", "rainbow", "bluetored", "staticblue", "rainbow", "bluetored"}, 5000,
```

Complete Isolation

- ✔ No model-specific names outside hdlo_models.cpp and sequences.cpp
- Framework code is 100% generic
- Easy to add/remove/modify models
- Clean separation of concerns

Registry System

- ✔ Automatic sequence cycling with individual timings

```
cpp

seq->startNewSequence("Fast", 30000, true); // 30 seconds
seq->startNewSequence("Slow", 90000, true); // 90 seconds
seq->startNewSequence("Medium", 60000, true); // 60 seconds
```

Workflow

1. Generate Model Data

```
Source Data → Auto-Generator → hdlo_models.h + hdlo_models.cpp
```

2. Create Choreography

```
Hand-edit sequences.cpp with creative timing and color choices
```

3. Upload and Run

```
Arduino IDE → Upload → LEDs cycle through sequences automatically
```

Adding a New Model

Step 1: Update Auto-Generated Files

hdlo_models.h:

```
cpp

const int nummodels = 7; // Was 6, now 7
extern colormodel* ourcolormodels[7]; // Update size
extern String modelNames[7]; // Update size
extern std::array<std::array<int, 6>, 120> newmodeldata; // Add declaration
```

hdlo_models.cpp:

```
cpp
```

```
// Add data array
std::array<std::array<int, 6>, 120> newmodeldata={{
    // ... 120 entries
}};

// Add to modelNames
String modelNames[7] = {
    "flowoctahedron",
    "flowoctahedroncompound1",
    "octachainshiftedby",
    "cycle",
    "cyclestemplate",
    "cycles",
    "newmodel" // New!
};

// Add to initialization
void initializemodels(){
    ourcolormodels[0] = new colormodel(flowoctahedrondata, modelNames[0]);
    // ... existing models
    ourcolormodels[6] = new colormodel(newmodeldata, modelNames[6]); // New!
}
```

Step 2: Use in Sequences (Optional)

sequences.cpp:

```
cpp

// Now you can reference it
seq->addStepByName(modelNames[6], {"dark", "rainbow"}, 5000, FADE, 1000);
```

Available Color Functions

Defined in `colorfunctions.h`:

- **"dark"** - All LEDs off (black)
- **"rainbow"** - Full HSV rainbow
- **"bluetored"** - Blue to red gradient
- **"staticblue"** - Solid blue

To add more colors:

1. Add function to `colorfunctions.h`
2. Add to `colorFunctionArray[]` in `colorfunctions.cpp`
3. Add name to `colorFunctionNames[]` in `colorfunctions.cpp`
4. Use in `sequences.cpp`

Transition Types

- **INSTANT**: Immediate switch (0ms transition)
- **FADE**: Cross-fade blend between steps
- **WIPE**: Sequential reveal across edges

Serial Monitor Output

Example output:

```
HDLO LED controller with model sequences
Registry has 4 sequences
Registry started with: Default Flow (5 steps)
Sequence: Default Flow | Step 1/5 - flowoctahedron
Sequence: Default Flow | Step 2/5 - flowoctahedroncompound1 (transitioning)
...
Switched to: Fast Pulse (6 steps)
Sequence: Fast Pulse | Step 1/6 - flowoctahedron
```

System Limits

Item	Limit
Maximum sequences	10
Maximum total steps	20
Edges per model	120
Values per edge	6
Color functions per step	1-6
Models	Unlimited*

*Limited only by memory

Files Changed from Original

New Files

- ✔ modelsequence.h
- ✔ modelsequence.cpp
- ✔ sequences.h
- ✔ sequences.cpp

Modified Files

- 🔧 hdlo_models.h (added modelNames export, removed initializeSequences)
- 🔧 hdlo_models.cpp (removed initializeSequences section)
- 🔧 hdlo_controller.ino (simplified, added sequences.h include)

Unchanged Files

- ✔ All other files remain as originally provided

Quick Reference

Define a Sequence

```
cpp

seq->startNewSequence("Name", duration_ms, enabled);
seq->addStepByName(modelNames[idx], {"colors"}, duration, TYPE, trans_dur);
```

Transition Types

```
cpp

INSTANT, 0    // No transition
FADE, 1000    // 1 second fade
WIPE, 1500    // 1.5 second wipe
```

Color Examples

```
cpp
```

```
{ "dark" } // 1 color
{ "dark", "rainbow" } // 2 colors
{ "dark", "rainbow", "bluetored", "staticblue" } // 4 colors
{ "dark", "rainbow", "bluetored", "staticblue", "rainbow", "bluetored" } // 6 colors
```

Success! ✨

The system is now:

-  Fully modular and maintainable
-  Ready for auto-generation of model data
-  Easy to create custom choreography
-  Completely isolated concerns
-  Supports 1-6 colors per step
-  No model-specific references in framework code